

Semana 6

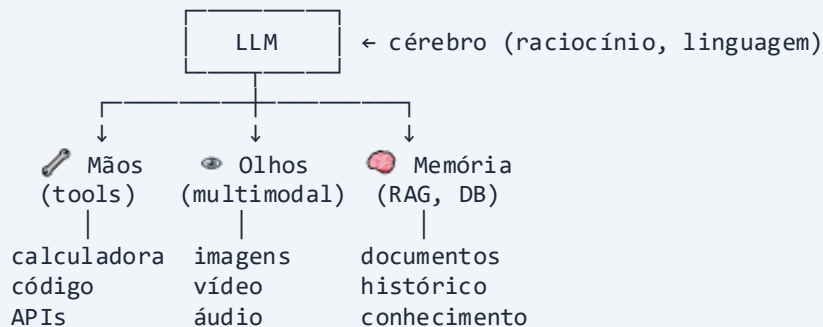
Structured Output e Tool Use

Damos mãos ao cérebro

*Semana 5: o LLM é stateless, sem ferramentas, sem memória.
Hoje: damos-lhe mãos — structured output e tool calling.*

SGAIM · 2025/2026 · MEIC · ISEL

Recap: o cérebro sem corpo



Semana 5: vimos o cérebro a funcionar — e o que lhe falta.

Hoje: as mãos. O LLM passa de “pensar” a “agir”.

O problema: texto livre

Pedem ao modelo: "Qual é o tempo em Lisboa amanhã?"

O tempo em Lisboa amanhã deverá ser agradável, com temperaturas a rondar os 18°C e céu parcialmente nublado, embora haja alguma possibilidade de chuva ao final do dia...

Como é que um programa extrai cidade=Lisboa, data=amanhã?

Regex? String matching? Frágil — muda o wording, parte tudo.

Precisamos de uma interface fiável entre o modelo e o sistema.

Versão 1 – JSON mode

Pedimos ao modelo: "Responde apenas em JSON."

```
{  
  "intent": "weather_query",  
  "confidence": 0.96  
}
```

O que ganhamos: parsing trivial com `json.loads()`.

Como funciona: durante a amostragem (softmax + temperatura), o sistema mascara tokens que produziam JSON inválido.

Após { → apenas " é permitido como próximo token significativo.

JSON mode controla sintaxe. Nada mais.

O que JSON mode NÃO garante

```
{  
  "city": "Lisboa",  
  "date": "amanhã",  
  "temperature": "está quentinho"  
}
```

JSON válido. Mas:

- Os campos podem mudar de nome entre chamadas
- Os tipos podem estar errados (temperature é string, não número)
- Valores semanticamente inválidos passam
- Campos obrigatórios podem não existir

JSON mode resolve o parsing. Não resolve a fiabilidade.

THINK

Imaginemos que o modelo devolve:

```
{ "name": "Alice", "age": "banana" }
```

Isto é JSON válido. Passaria JSON mode.

Porque é que isto não serve para automação?
O que falta?

Pensem 1 minuto.

Versão 2 — Structured output

Definimos um schema — um contrato:

```
from pydantic import BaseModel, Field

class WeatherArgs(BaseModel):
    city: str = Field(..., min_length=1)
    date: str = Field(..., min_length=1)
```

O output do modelo tem de obedecer ao schema.
Se não obedecer → validação falha → retry ou erro.

Structured output = JSON mode + disciplina de schema + validação.

O que muda com structured output

Sem schema	Com schema
Campos podem aparecer ou não	Campos obrigatórios verificados
Tipos não verificados	city: str, date: str — tipados
Valores livres	Validação: min_length=1, ranges, enums
O modelo decide o formato	O sistema impõe o formato
Erros só aparecem a jusante	Erros detectados imediatamente

Regra: A validação é trabalho do sistema, não do modelo.

O modelo propõe. O sistema valida. Se falhar, rejeita ou pede outra vez.

Versão 3 – Tool calling

Structured output formata dados. Tool calling vai além: o modelo escolhe uma acção.

```
{  
  "action": "call_tool",  
  "tool_name": "get_weather",  
  "arguments": {"city": "Lisboa", "date": "amanhã"}  
}
```

O modelo já não só formata — decide:

- Responder directamente? → "action": "respond"
- Chamar uma ferramenta? → "action": "call_tool"

Tool calling = structured output + selecção de acção + execução externa.

A fórmula compacta

JSON mode = controlo de sintaxe
Structured output = controlo de sintaxe + schema
Tool calling = controlo de sintaxe + schema + comportamento

De texto livre a decisões executáveis em 3 passos.

Sintaxe

o output é parseable

Schema

o output obedece a um contrato

Comportamento

o output é uma acção escolhida

Esta progressão é a mesma em qualquer framework — OpenAI, Anthropic, Ollama.

O schema de decisão

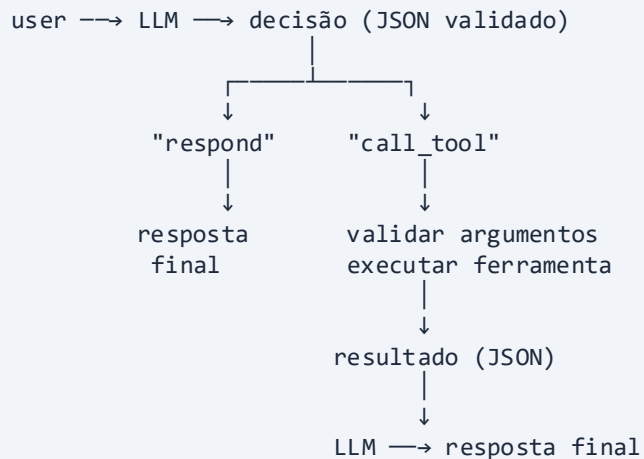
```
class AgentDecision(BaseModel):  
    action: Literal["respond", "call_tool"]  
    answer: Optional[str] = None           # se action == "respond"  
    tool_name: Optional[str] = None        # se action == "call_tool"  
    arguments: Optional[Dict] = None       # se action == "call_tool"
```

Validação cruzada:

- "respond" → answer obrigatório, sem tool_name nem arguments
- "call_tool" → tool_name + arguments obrigatórios, sem answer

O schema não é documentação — é o contrato que o sistema impõe em cada chamada.

O loop de orquestração



O modelo não executa nada. Propõe uma acção. O sistema valida, executa, e devolve o resultado.

Exemplo completo: uma pergunta sobre o tempo

1. user: "Que tempo faz em Lisboa amanhã?"
2. LLM → JSON:

```
{"action": "call_tool", "tool_name": "get_weather",  
  "arguments": {"city": "Lisboa", "date": "amanhã"}}
```
3. Sistema: valida → OK. Executa `get_weather("Lisboa", "amanhã")`
4. Ferramenta → resultado:

```
{"city": "Lisboa", "date": "amanhã",  
  "temperature_c": 17, "condition": "partly cloudy"}
```
5. Sistema envia resultado ao LLM
6. LLM → resposta final:
"Em Lisboa amanhã: 17°C com céu parcialmente nublado."

O LLM fez duas coisas: decidiu chamar a ferramenta (passo 2) e formulou a resposta (passo 6). O sistema fez tudo o resto.

THINK

No loop de orquestração, duas entidades trabalham juntas.

Classifiquem: trabalho do modelo ou do sistema?

1. Escolher qual ferramenta usar
2. Validar se o JSON obedece ao schema
3. Executar a ferramenta (chamar API, fazer cálculo)
4. Extrair argumentos do pedido do utilizador
5. Decidir se precisa de ferramenta ou pode responder directamente
6. Gerir o retry se a validação falhar

Pensem 1 minuto.

Modelo vs sistema — a separação

Tarefa	Quem
Escolher ferramenta	Modelo — selecção semântica
Validar JSON/schema	Sistema — Pydantic, JSON Schema
Executar ferramenta	Sistema — código Python
Extrair argumentos	Modelo — parsing semântico
Decidir tool vs resposta	Modelo — política de acção
Gerir retries	Sistema — loop de controlo

O comportamento "agente" não vive só dentro do modelo — emerge da interacção entre modelo e infraestrutura.

SFT e tool calling

Um modelo base (pré-treinado) pode produzir JSON se o prompt for bom.
Mas o SFT (aula 04) torna-o muito mais fiável:

Sem SFT	Com SFT
JSON por vezes mal formado	JSON consistente
Escolha de ferramenta errática	Seleccção robusta
Argumentos incompletos	Preenchimento preciso
Não sabe quando NÃO chamar	Abstém-se quando adequado

É por isso que modelos "instruct" existem — são modelos base + SFT para seguir instruções, produzir formatos, e usar ferramentas.

Quando usam llama3.2:3b (e não llama3.2:3b-text), estão a usar um modelo com SFT.

Attention no tool calling

O mecanismo de attention — que implementaram no ex3 — é crucial aqui:

"Que tempo faz em Lisboa amanhã?"



`get_weather(city, date)`

O modelo alinha:

- O pedido do utilizador ↔ a descrição da ferramenta
- As palavras "Lisboa" e "amanhã" ↔ os parâmetros city e date
- O resultado da ferramenta ↔ a resposta em linguagem natural

Tool calling parece muito com parsing semântico com semântica de acção.
O mesmo attention de sempre — mas agora a serviço de decisões executáveis.

Wrap-up

Hoje:

- Progressão: texto livre → JSON mode → structured output → tool calling
- O schema como contrato entre modelo e sistema
- O loop de orquestração: LLM → tool → resultado → LLM
- Separação fundamental: o modelo propõe, o sistema executa

Semana 7: RAG

- Porquê o LLM alucina — e como o RAG resolve
- Embeddings e similaridade semântica
- Vector databases (Chroma, FAISS)
- Pipeline: ingest → chunk → embed → store → retrieve → generate

TPC:

- Completar OllamaCompatibleClient
- Adicionar 1+ ferramenta nova ao stack
- Experimentar: schema mais complexo → o modelo continua fiável?

O LLM ganha mãos — pode agir no mundo. Mas o sistema é quem valida e executa.